

Dynamic Programming (DP) Basics

Covers: Recursion → Memoization, Tabulation, State Definition, Transition, Base Cases, Space Optimization, 1D DP, 2D DP

1. Overview

Dynamic Programming is a technique for solving problems by breaking them into overlapping subproblems, solving each subproblem once, and storing the results for reuse.

DP sits between **divide-and-conquer** (where subproblems are independent) and **brute-force recursion** (where subproblems are recomputed many times). The key insight: *if the same subproblem appears multiple times, compute it once and remember the answer.*

DP is built on two fundamental directions:

Approach	Direction	How it works
Memoization (Top-Down)	Recursion + Cache	Solve recursively, cache results before returning.
Tabulation (Bottom-Up)	Iterative + Table	Fill a table from smallest subproblems upward.

The eight sub-topics covered here:

#	Topic	Essence
1	Recursion → Memoization	Start with a recursive brute force, add a cache.
2	Tabulation	Build the DP table iteratively from base cases.
3	State Definition	What does <code>dp[i][j]</code> represent?
4	Transition	How to compute <code>dp[i][j]</code> from smaller states.
5	Base Cases	The smallest subproblems that seed the table.
6	Space Optimization	Reduce $O(n^2)$ memory to $O(n)$ or $O(1)$.
7	1D DP	Single-dimension state (Fibonacci, LIS, House Robber).
8	2D DP	Two-dimension state (LCS, Edit Distance, Knapsack).

2. Intuition

Core Idea

Every DP problem asks: "Can I compute the answer for a larger input using answers I already computed for smaller inputs?"

Analogy — Climbing a staircase:

- You want to reach step **N**.
- You can take 1 step or 2 steps at a time.
- The number of ways to reach step **N** = ways to reach step **N-1** + ways to reach step **N-2**.
- Once you compute **ways[5]**, you never recompute it — you just look it up.
- This is the heart of DP: **store, don't recompute**.

Recursion → Memoization Intuition

1. Write a recursive function that solves the problem.
2. In the function, before recursing, check if you've already solved this state.
3. If yes, return the cached value. If no, compute recursively and cache.

```
fib(n):
    if n is in cache: return cache[n]
    cache[n] = fib(n-1) + fib(n-2)
    return cache[n]
```

This is **top-down** because you start from the original problem and go down to base cases.

Tabulation Intuition

1. Identify the smallest subproblems (base cases).
2. Fill a table iteratively from smallest to largest.
3. The answer is the final cell.

```
fib:
    dp[0] = 0, dp[1] = 1
    for i = 2 to n:
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

This is **bottom-up** because you start from the bottom (base cases) and build up.

State Definition Intuition

The **state** is the set of parameters that uniquely describe a subproblem. Everything needed to compute the answer must be captured in the state.

- **Good state:** **dp[i]** = max profit using first **i** items.
- **Bad state:** **dp[i]** = max profit (missing information about weight/capacity).

Transition Intuition

The **transition** is the formula that relates **dp[state]** to smaller **dp[smaller_state]**. It answers: *"How do I compute the current state from previous states?"*

```
dp[i] = max(dp[i-1], dp[i-2] + value[i])
```

Base Cases Intuition

Base cases are the smallest subproblems you can solve directly without recursion. They are the foundation of the DP table.

- $dp[0] = 0$ (no items $\rightarrow$ no profit)
- $dp[1] = arr[0]$ (single element $\rightarrow$ that element)

Space Optimization Intuition

Many DP problems only need the last few states. Instead of storing the entire table, store only the relevant rows/columns.

- Fibonacci: Only need last 2 values $\rightarrow O(1)$ space.
- 2D DP: Only need previous row + current row $\rightarrow O(n)$ space.

1D DP Intuition

State is a single integer i (position, amount, length). Table is a 1D array.

2D DP Intuition

State is two integers (i, j) (positions in two strings, index + remaining capacity). Table is a 2D matrix.

3. When to Use It

Use DP when the problem has these two properties:

✓ Optimal Substructure

The optimal solution of the problem can be constructed from optimal solutions of its subproblems.

"The best way to solve the whole thing includes the best way to solve a smaller thing."

✓ Overlapping Subproblems

The same subproblems appear multiple times in the recursion tree.

"The same smaller problem is solved again and again from different paths."

Trigger Phrases from Problems

Phrase	Likely DP
"Maximum/minimum sum/ways to reach..."	DP
"Number of ways to..."	Counting DP

Phrase	Likely DP
"Longest/shortest subsequence..."	LCS, LIS variants
"Knapsack/capacity/weight"	0/1 Knapsack DP
"Edit distance / convert string A to B"	2D DP (Edit Distance)
"House robber / adjacent"	1D DP with state machine
"Grid path / robot moving right/down"	2D Grid DP
"Subset sum / partition equal subset"	Subset DP
"Stock prices with transactions"	State-machine DP
"Break word / concatenate"	1D DP (Word Break)
"Palindrome partitioning"	2D DP
"Matrix chain multiplication"	Interval DP (2D)

Checklist (ask yourself)

1. Can I brute-force with recursion? (Yes → DP candidate)
2. Does the recursion revisit the same states? (Yes → DP needed)
3. Can I define `dp[i]` or `dp[i][j]` meaningfully? (Yes → proceed)
4. Is there a recurrence relation? (Yes → DP is correct)

4. When Not to Use It

✗ Greedy Works

If picking the locally optimal choice always yields the global optimum, greedy is simpler and faster. DP is overkill.

Example: Coin change with unlimited coins — if coin denominations are canonical (1, 5, 10, 25), greedy works. Use DP only when greedy fails.

✗ Subproblems Are Independent

If subproblems don't overlap, plain recursion or divide-and-conquer is fine. DP adds cache overhead for no benefit.

Example: Merge sort — subarrays don't overlap. No DP.

✗ State Space Is Too Large

If the state has too many dimensions or each dimension is too large, DP becomes infeasible.

Example: DP with state `(mask, i, j, k)` where `mask` is 2^{20} — too large.

✗ Simpler Alternatives Exist

- **Sliding window** for subarray sum problems (instead of DP).
- **Two-pointer** for some string problems.
- **Binary search** for some optimization problems.
- **BFS** for shortest path in unweighted graphs (simpler than DP).

✗ Wrong Assumptions

- **"DP always gives the best answer"** — Yes, but only if the state captures all constraints. If you miss a dimension, the answer is wrong.
- **"DP is always $O(n^2)$ "** — Some DP is $O(n^3)$ or exponential.
- **"Memoization and tabulation are interchangeable"** — Not always. Tabulation can be faster (no recursion overhead) and easier to optimize for space.

✗ Edge Cases Where DP Fails / Is Inefficient

- **Negative cycles** in shortest path — DP (Bellman-Ford) works but detects cycles, doesn't find shortest path.
- **Large constraints** — $n = 10^5$ with $O(n^2)$ DP is too slow.
- **Non-linear transitions** — If transition depends on complex conditions, DP may become unwieldy.

5. Core Concepts

5.1 Recursion → Memoization

What it is: Start with a recursive brute-force solution. Add a cache (dictionary/array) keyed by the state parameters. Before computing, check the cache; after computing, store in cache.

Why it matters: It's the most natural way to solve DP if you can write the recursion. No need to think about table-filling order.

Example:

```
// Without memoization -  $O(2^n)$ 
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}

// With memoization -  $O(n)$ 
int fib(int n, vector<int>& memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n-1, memo) + fib(n-2, memo);
}
```

5.2 Tabulation

What it is: Build a DP table iteratively. Start from base cases. Fill the table in a specific order (usually increasing index). The answer is `dp[n]` or `dp[n][m]`.

Why it matters: Usually faster than memoization (no recursion overhead). Easier to space-optimize. Avoids stack overflow for deep recursion.

Example:

```
int fib(int n) {
    vector<int> dp(n+1);
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

5.3 State Definition

What it is: The state is the set of parameters that uniquely identify a subproblem. The state definition determines what `dp[...]` stores.

Why it matters: A wrong state definition leads to wrong answers or exponential time. A good state definition makes the transition obvious.

Examples:

Problem	State	Meaning
Fibonacci	<code>dp[i]</code>	i-th Fibonacci number
0/1 Knapsack	<code>dp[i][w]</code>	Max value using first i items with capacity w
LCS	<code>dp[i][j]</code>	LCS length of first i chars of A and first j of B
LIS	<code>dp[i]</code>	Length of LIS ending at index i
Edit Distance	<code>dp[i][j]</code>	Min edits to convert A[0..i] to B[0..j]
Coin Change	<code>dp[i]</code>	Min coins needed to make amount i

Guidelines for defining state:

- Start with what changes during recursion — those are your state parameters.
- Keep the state as small as possible (fewer dimensions → smaller table).
- Every parameter needed to compute the answer must be in the state.

5.4 Transition

What it is: The recurrence relation that computes `dp[state]` from smaller states.

Why it matters: The transition is the core of the DP solution. Everything else is scaffolding.

How to derive it:

1. Ask: "What choices do I have at this state?"
2. For each choice, ask: "What smaller state does this lead to?"
3. Combine: $dp[state] = \text{best/union of } dp[\text{smaller_state}] + \text{cost_of_choice}$

Examples:

Problem	Transition
Fibonacci	$dp[i] = dp[i-1] + dp[i-2]$
0/1 Knapsack	$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])$
LCS	$dp[i][j] = (A[i]==B[j]) ? 1+dp[i-1][j-1] : \max(dp[i-1][j], dp[i][j-1])$
House Robber	$dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$
Coin Change	$dp[i] = \min(dp[i], dp[i-coin] + 1)$

5.5 Base Cases

What it is: The smallest subproblems that can be solved directly without recurrence.

Why it matters: Without correct base cases, the DP table has no foundation. Wrong base cases → wrong answers.

How to find them:

- What happens when all parameters are 0 or empty?
- What happens with 1 element?
- What happens at boundaries?

Examples:

Problem	Base Cases
Fibonacci	$dp[0] = 0, dp[1] = 1$
0/1 Knapsack	$dp[0][w] = 0$ for all w (no items)
LCS	$dp[0][j] = 0, dp[i][0] = 0$ (empty string)
LIS	$dp[i] = 1$ for all i (each element alone is LIS of length 1)
Coin Change	$dp[0] = 0$ (0 coins needed for amount 0)
Edit Distance	$dp[i][0] = i, dp[0][j] = j$ (delete/insert all)

5.6 Space Optimization

What it is: Reducing memory usage by noticing that the DP table only needs a few rows/columns at a time, not the entire table.

Why it matters: Can reduce memory from $O(n^2)$ to $O(n)$, or $O(n)$ to $O(1)$. This matters for large constraints ($n = 10^5$).

Common patterns:

Problem	Full Space	Optimized	How
Fibonacci	$O(n)$	$O(1)$	Only need last 2 values
0/1 Knapsack	$O(n \times W)$	$O(W)$	Only need previous row
LCS	$O(n \times m)$	$O(\min(n,m))$	Only need previous row
Grid DP	$O(n \times m)$	$O(m)$	Only need previous row
House Robber	$O(n)$	$O(1)$	Only need last 2 states

Key insight: If $dp[i]$ only depends on $dp[i-1]$ and $dp[i-2]$, you only need to keep the last 2 values. If $dp[i][j]$ only depends on $dp[i-1][*]$ and $dp[i][*]$, you only need the previous row and current row.

5.7 1D DP

What it is: DP where the state is a single integer, so the table is a 1D array.

Why it matters: Foundational. Many problems (Fibonacci, LIS, House Robber, Coin Change, Word Break) are 1D DP.

When to use: When the subproblem can be described by a single parameter (index, amount, length).

Common patterns:

- Linear DP: $dp[i]$ depends on $dp[i-1]$, $dp[i-2]$, etc.
- LIS: $dp[i]$ depends on all $dp[j]$ where $j < i$.
- Subset DP: $dp[sum]$ depends on $dp[sum - num]$.

5.8 2D DP

What it is: DP where the state is two integers, so the table is a 2D matrix.

Why it matters: Many classic problems (LCS, Edit Distance, 0/1 Knapsack, longest palindrome substring) are 2D DP.

When to use: When the subproblem needs two parameters — e.g., positions in two strings, or index + remaining capacity.

Common patterns:

- String DP: $dp[i][j]$ — first i chars of A, first j of B.
- Grid DP: $dp[i][j]$ — position in grid.
- Knapsack: $dp[i][w]$ — first i items, capacity w .
- Interval DP: $dp[i][j]$ — substring from i to j .

6. Step-by-Step Algorithm

General DP Problem-Solving Framework

Step 1: Identify the problem type

- Is it optimization (max/min)? Counting? Feasibility?
- Look for overlapping subproblems and optimal substructure.

Step 2: Define the state

- What parameters uniquely identify a subproblem?
- Write: `dp[i] = ...` or `dp[i][j] = ...`
- Start with what changes in the recursive version.

Step 3: Write the recurrence (transition)

- How does `dp[state]` relate to smaller states?
- Write the formula.
- Verify it on a small example.

Step 4: Identify base cases

- What are the smallest subproblems?
- What are the boundary conditions?
- Initialize the DP table for these.

Step 5: Determine the order of computation

- For 1D: usually left to right (increasing *i*).
- For 2D: usually increasing *i*, increasing *j* (or both).
- For interval DP: increasing length.

Step 6: Compute the answer

- The answer is usually `dp[n]`, `dp[n][m]`, or `min/max of dp[n][*]`.

Step 7: (Optional) Space optimization

- Can we keep only the last few rows/values?
- Implement the optimized version.

Recursion → Memoization Steps

1. Write a recursive function `solve(params)`.
2. Add a cache (dictionary or 1D/2D array initialized to -1).
3. At the start of `solve`, check if `cache[params]` is computed.
4. If yes, return `cache[params]`.
5. If no, compute recursively, store in cache, and return.
6. Call `solve(original_params)`.

Tabulation Steps

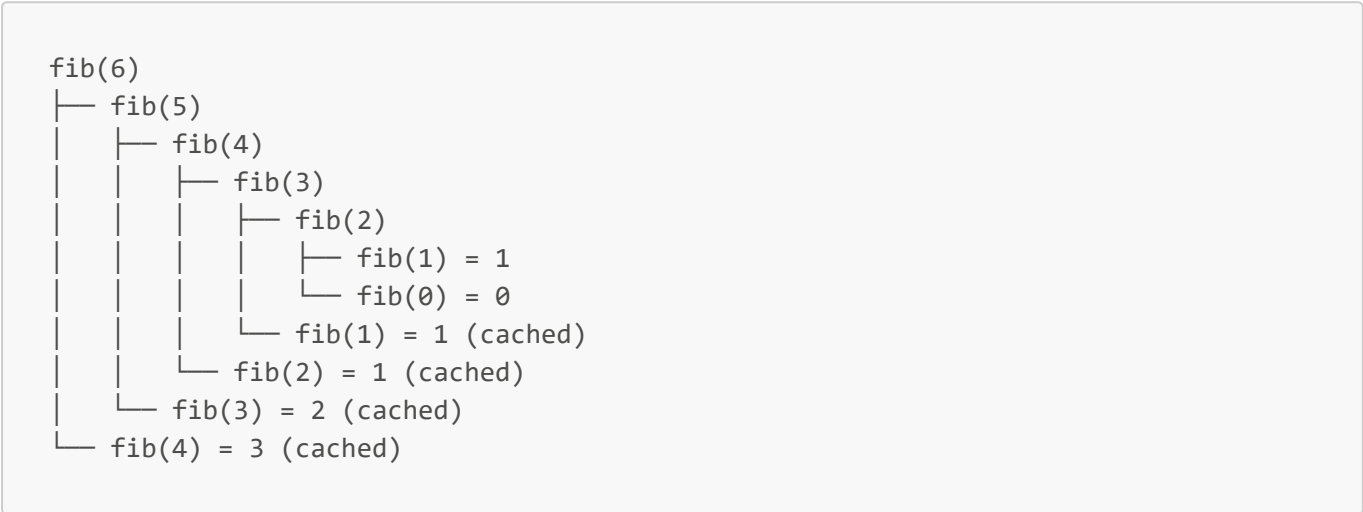
1. Create a DP array/matrix of appropriate size.

- 2. Initialize base cases.
- 3. Iterate over all states in increasing order of size.
- 4. For each state, compute using the transition formula.
- 5. Return the required cell.

7. Dry Run

Example: Fibonacci (n = 6)

Memoization



Call	Cache Hit?	Value	Cache State
fib(6)	No	compute	-
fib(5)	No	compute	-
fib(4)	No	compute	-
fib(3)	No	compute	-
fib(2)	No	compute	-
fib(1)	No	1	memo[1]=1
fib(0)	No	0	memo[0]=0
fib(2)	compute	1+0=1	memo[2]=1
fib(3)	compute	1+1=2	memo[3]=2
fib(4)	compute	2+1=3	memo[4]=3
fib(5)	compute	3+2=5	memo[5]=5
fib(6)	compute	5+3=8	memo[6]=8

Result: fib(6) = 8

Tabulation

i	dp[i]	Computation
0	0	Base case
1	1	Base case
2	1	$dp[1] + dp[0] = 1 + 0$
3	2	$dp[2] + dp[1] = 1 + 1$
4	3	$dp[3] + dp[2] = 2 + 1$
5	5	$dp[4] + dp[3] = 3 + 2$
6	8	$dp[5] + dp[4] = 5 + 3$

Result: $dp[6] = 8$

Example: 0/1 Knapsack ($n=3$, $W=5$)

Items: (value, weight) = [(60, 3), (50, 2), (70, 4)]

DP Table: $dp[i][w]$ = max value with first i items and capacity w

i\w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	60	60	60
2	0	0	50	60	60	110
3	0	0	50	60	70	110

Computation details:

- $dp[1][3]$: item 1 ($w=3$, $v=60$) fits $\rightarrow \max(0, 0+60) = 60$
- $dp[2][5]$: item 2 ($w=2$, $v=50$) fits $\rightarrow \max(60, 60+50) = 110$
- $dp[3][4]$: item 3 ($w=4$, $v=70$) fits $\rightarrow \max(60, 0+70) = 70$

Result: $dp[3][5] = 110$

Example: LCS of "ABC" and "AC"

DP Table: $dp[i][j]$ = LCS of first i chars of A and first j of B

i\j	0	1(A)	2(C)
0	0	0	0
1(A)	0	1	1
2(B)	0	1	1
3(C)	0	1	2

Computation details:

- `dp[1][1]`: $A[0]=A, B[0]=A$, match $\rightarrow 1 + dp[0][0] = 1$
- `dp[3][2]`: $A[2]=C, B[1]=C$, match $\rightarrow 1 + dp[2][1] = 1 + 1 = 2$

Result: LCS length = 2 ("AC")

8. C++ Implementation

8.1 Fibonacci — Memoization vs Tabulation vs Space Optimized

```
#include <bits/stdc++.h>
using namespace std;

// ----- MEMOIZATION (Top-Down) -----
int fibMemo(int n, vector<int>& memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

int fibMemo(int n) {
    vector<int> memo(n + 1, -1);
    return fibMemo(n, memo);
}

// ----- TABULATION (Bottom-Up) -----
int fibTab(int n) {
    if (n <= 1) return n;
    vector<int> dp(n + 1);
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}

// ----- SPACE OPTIMIZED -----
int fibOpt(int n) {
    if (n <= 1) return n;
    int prev2 = 0, prev1 = 1;
    for (int i = 2; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

8.2 0/1 Knapsack — 2D DP + Space Optimized

```

#include <bits/stdc++.h>
using namespace std;

// ----- 2D TABULATION -----
// Returns max value for capacity W with items (val, wt)
int knapsack2D(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i - 1] <= w) {
                // Include item i-1
                dp[i][w] = max(dp[i - 1][w],
                               dp[i - 1][w - wt[i - 1]] + val[i - 1]);
            } else {
                // Exclude item i-1
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[n][W];
}

// ----- SPACE OPTIMIZED (1D) -----
int knapsack1D(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; i++) {
        // Traverse backwards to avoid reusing the same item
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
        }
    }
    return dp[W];
}

```

8.3 LCS (Longest Common Subsequence) — 2D + Space Optimized

```

#include <bits/stdc++.h>
using namespace std;

// ----- 2D TABULATION -----
int lcs2D(string& a, string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = 1 + dp[i - 1][j - 1];
        }
    }
}

```

```

        else
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    }
}
return dp[n][m];
}

// ----- SPACE OPTIMIZED -----
int lcs1D(string& a, string& b) {
    int n = a.size(), m = b.size();
    vector<int> prev(m + 1, 0), curr(m + 1, 0);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])
                curr[j] = 1 + prev[j - 1];
            else
                curr[j] = max(prev[j], curr[j - 1]);
        }
        swap(prev, curr);
    }
    return prev[m];
}

```

8.4 LIS (Longest Increasing Subsequence) — $O(n^2)$ and $O(n \log n)$

```

#include <bits/stdc++.h>
using namespace std;

// -----  $O(n^2)$  DP -----
int lisN2(vector<int>& arr) {
    int n = arr.size();
    vector<int> dp(n, 1);
    int ans = 1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (arr[j] < arr[i])
                dp[i] = max(dp[i], dp[j] + 1);
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}

// -----  $O(n \log n)$  using patience sorting -----
int lisNlogN(vector<int>& arr) {
    vector<int> tails;
    for (int x : arr) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end())

```

```

        tails.push_back(x);
    else
        *it = x;
    }
    return tails.size();
}

```

8.5 House Robber — 1D DP + Space Optimized

```

#include <bits/stdc++.h>
using namespace std;

// ----- 1D TABULATION -----
int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];

    vector<int> dp(n);
    dp[0] = nums[0];
    dp[1] = max(nums[0], nums[1]);

    for (int i = 2; i < n; i++)
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i]);

    return dp[n - 1];
}

// ----- SPACE OPTIMIZED (O(1)) -----
int robOpt(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];

    int prev2 = nums[0];
    int prev1 = max(nums[0], nums[1]);

    for (int i = 2; i < n; i++) {
        int curr = max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

```

8.6 Coin Change (Minimum Coins) — 1D DP

```

#include <bits/stdc++.h>
using namespace std;

```

```

int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i && dp[i - coin] != INT_MAX)
                dp[i] = min(dp[i], 1 + dp[i - coin]);
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}

```

8.7 Edit Distance (Levenshtein Distance) — 2D + Space Optimized

```

#include <bits/stdc++.h>
using namespace std;

// ----- 2D TABULATION -----
int editDistance2D(string& a, string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    // Base cases
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1];
            else
                dp[i][j] = 1 + min({dp[i - 1][j],          // delete
                                   dp[i][j - 1],          // insert
                                   dp[i - 1][j - 1]});      // replace
        }
    }
    return dp[n][m];
}

// ----- SPACE OPTIMIZED -----
int editDistance1D(string& a, string& b) {
    int n = a.size(), m = b.size();
    vector<int> prev(m + 1), curr(m + 1);

    for (int j = 0; j <= m; j++) prev[j] = j;

    for (int i = 1; i <= n; i++) {
        curr[0] = i;
        for (int j = 1; j <= m; j++) {

```

```

        if (a[i - 1] == b[j - 1])
            curr[j] = prev[j - 1];
        else
            curr[j] = 1 + min({prev[j], curr[j - 1], prev[j - 1]});
    }
    swap(prev, curr);
}
return prev[m];
}

```

9. Python Implementation

9.1 Fibonacci — All Three Versions

```

# ----- MEMOIZATION -----
def fib_memo(n: int, memo: list = None) -> int:
    if memo is None:
        memo = [-1] * (n + 1)
    if n <= 1:
        return n
    if memo[n] != -1:
        return memo[n]
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

# ----- TABULATION -----
def fib_tab(n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

# ----- SPACE OPTIMIZED -----
def fib_opt(n: int) -> int:
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for _ in range(2, n + 1):
        curr = prev1 + prev2
        prev2, prev1 = prev1, curr
    return prev1

```

9.2 0/1 Knapsack

```
# ----- 2D TABULATION -----
def knapsack_2d(W: int, wt: list, val: list, n: int) -> int:
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            if wt[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w],
                               dp[i - 1][w - wt[i - 1]] + val[i - 1])
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][W]

# ----- SPACE OPTIMIZED -----
def knapsack_1d(W: int, wt: list, val: list, n: int) -> int:
    dp = [0] * (W + 1)

    for i in range(n):
        for w in range(W, wt[i] - 1, -1):
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i])
    return dp[W]
```

9.3 LCS

```
# ----- 2D -----
def lcs_2d(a: str, b: str) -> int:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[n][m]

# ----- SPACE OPTIMIZED -----
def lcs_1d(a: str, b: str) -> int:
    n, m = len(a), len(b)
    prev = [0] * (m + 1)

    for i in range(1, n + 1):
        curr = [0] * (m + 1)
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                curr[j] = 1 + prev[j - 1]
            else:
                curr[j] = max(curr[j - 1], prev[j])
        prev = curr
    return prev[m]
```

```

        curr[j] = max(prev[j], curr[j - 1])
    prev = curr
    return prev[m]

```

9.4 LIS

```

# ----- O(n²) -----
def lis_n2(arr: list) -> int:
    n = len(arr)
    dp = [1] * n
    ans = 1

    for i in range(n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)
        ans = max(ans, dp[i])
    return ans

# ----- O(n log n) -----
import bisect

def lis_nlogn(arr: list) -> int:
    tails = []
    for x in arr:
        idx = bisect.bisect_left(tails, x)
        if idx == len(tails):
            tails.append(x)
        else:
            tails[idx] = x
    return len(tails)

```

9.5 House Robber

```

# ----- 1D DP -----
def rob(nums: list) -> int:
    n = len(nums)
    if n == 0:
        return 0
    if n == 1:
        return nums[0]

    dp = [0] * n
    dp[0] = nums[0]
    dp[1] = max(nums[0], nums[1])

    for i in range(2, n):
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])

```

```

    return dp[n - 1]

# ----- SPACE OPTIMIZED -----
def rob_opt(nums: list) -> int:
    n = len(nums)
    if n == 0:
        return 0
    if n == 1:
        return nums[0]

    prev2 = nums[0]
    prev1 = max(nums[0], nums[1])

    for i in range(2, n):
        curr = max(prev1, prev2 + nums[i])
        prev2, prev1 = prev1, curr

    return prev1

```

9.6 Coin Change

```

def coin_change(coins: list, amount: int) -> int:
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i and dp[i - coin] != float('inf'):
                dp[i] = min(dp[i], 1 + dp[i - coin])

    return -1 if dp[amount] == float('inf') else dp[amount]

```

9.7 Edit Distance

```

# ----- 2D -----
def edit_distance_2d(a: str, b: str) -> int:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:

```

```

        dp[i][j] = dp[i - 1][j - 1]
    else:
        dp[i][j] = 1 + min(dp[i - 1][j],      # delete
                           dp[i][j - 1],      # insert
                           dp[i - 1][j - 1])    # replace

    return dp[n][m]

# ----- SPACE OPTIMIZED -----
def edit_distance_1d(a: str, b: str) -> int:
    n, m = len(a), len(b)
    prev = list(range(m + 1))

    for i in range(1, n + 1):
        curr = [0] * (m + 1)
        curr[0] = i
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                curr[j] = prev[j - 1]
            else:
                curr[j] = 1 + min(prev[j], curr[j - 1], prev[j - 1])
        prev = curr
    return prev[m]

```

10. Code Explanation

10.1 Fibonacci — Memoization

```

vector<int> memo(n + 1, -1); // Initialize cache with -1 (uncomputed)
if (n <= 1) return n;       // Base case: fib(0)=0, fib(1)=1
if (memo[n] != -1) return memo[n]; // Return cached result if computed
return memo[n] = fibMemo(n-1) + fibMemo(n-2); // Compute, cache, return

```

Why this works:

- The cache ensures each `fib(k)` is computed once.
- The recursion tree collapses from $O(2^n)$ to $O(n)$ calls.
- The `-1` sentinel distinguishes "not computed yet" from actual values.

10.2 Fibonacci — Tabulation

```

dp[0] = 0; dp[1] = 1; // Base cases
for (int i = 2; i <= n; i++)
    dp[i] = dp[i-1] + dp[i-2]; // Transition: current = sum of previous two
return dp[n];

```

Why this works:

- Iterative filling ensures all dependencies are computed before use.
- No recursion overhead, no stack overflow risk.
- Order is natural: smaller indices first.

10.3 0/1 Knapsack — 2D

```
for (int i = 1; i <= n; i++) {           // For each item
    for (int w = 0; w <= W; w++) {       // For each capacity
        if (wt[i-1] <= w)               // If item fits
            dp[i][w] = max(dp[i-1][w], // Exclude item
                           dp[i-1][w - wt[i-1]] + val[i-1]); // Include item
        else
            dp[i][w] = dp[i-1][w];       // Can't include, carry forward
    }
}
```

Key insight: The transition has two choices — exclude the item (keep previous row's value) or include it (add its value to the best with reduced capacity). The `max` picks the better option.

10.4 0/1 Knapsack — Space Optimized (1D)

```
for (int i = 0; i < n; i++) {           // For each item
    for (int w = W; w >= wt[i]; w--) { // Traverse backwards!
        dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    }
}
```

Why backwards: If we traverse forward, `dp[w - wt[i]]` might already include the current item (since we updated it in the same iteration), effectively allowing multiple uses of the same item. Backwards traversal prevents this — each item is considered at most once.

10.5 LCS

```
if (a[i-1] == b[j-1])
    dp[i][j] = 1 + dp[i-1][j-1];       // Characters match, extend LCS
else
    dp[i][j] = max(dp[i-1][j],         // Skip char from A
                   dp[i][j-1]);        // Skip char from B
```

Why this works:

- If characters match, they can be part of the LCS, so we add 1 to the LCS of the prefixes without these characters.
- If they don't match, we take the best of skipping either character.

10.6 LIS — $O(n^2)$

```

dp[i] = 1; // Each element alone is LIS of length 1
for (int j = 0; j < i; j++) {
    if (arr[j] < arr[i]) // If j can come before i in increasing
        order
        dp[i] = max(dp[i], dp[j] + 1); // Extend the LIS ending at j
}

```

Why this works: `dp[i]` stores the length of the longest increasing subsequence that **ends at** index `i`. For each `j < i` with `arr[j] < arr[i]`, we can extend the LIS ending at `j` by including `arr[i]`.

10.7 House Robber

```

dp[0] = nums[0]; // Only one house
dp[1] = max(nums[0], nums[1]); // Two houses: pick the richer one
dp[i] = max(dp[i-1], dp[i-2] + nums[i]); // Skip or rob current house

```

Why this works: At each house, you have two choices:

- **Skip** the current house → profit = best up to previous house (`dp[i-1]`)
- **Rob** the current house → profit = current house + best up to two houses ago (`dp[i-2] + nums[i]`)

The `max` picks the better option.

10.8 Coin Change

```

dp[0] = 0; // 0 coins needed for amount 0
for (int i = 1; i <= amount; i++) {
    for (int coin : coins) {
        if (coin <= i && dp[i - coin] != INT_MAX)
            dp[i] = min(dp[i], 1 + dp[i - coin]); // Use one coin of this
        denomination
    }
}

```

Why this works: For each amount `i`, we try every coin. If we use a coin of value `coin`, we need 1 coin plus the minimum coins needed for amount `i - coin`. We take the minimum over all coins.

10.9 Edit Distance

```

// Base cases
dp[i][0] = i; // Delete all i characters from A
dp[0][j] = j; // Insert all j characters into A

// Transition
if (a[i-1] == b[j-1])

```

```
    dp[i][j] = dp[i-1][j-1]; // No operation needed
else
    dp[i][j] = 1 + min({
        dp[i-1][j],      // Delete a[i-1]
        dp[i][j-1],      // Insert b[j-1] into A
        dp[i-1][j-1]     // Replace a[i-1] with b[j-1]
    });
```

Why this works: Three operations are possible:

- **Delete:** Remove `a[i-1]` → cost = 1 + edit distance of `A[0..i-2]` and `B[0..j-1]`
- **Insert:** Add `b[j-1]` to A → cost = 1 + edit distance of `A[0..i-1]` and `B[0..j-2]`
- **Replace:** Change `a[i-1]` to `b[j-1]` → cost = 1 + edit distance of `A[0..i-2]` and `B[0..j-2]`

11. Complexity Analysis

General Table

Algorithm	Time	Space (Full)	Space (Optimized)
Fibonacci (Memo)	$O(n)$	$O(n)$	—
Fibonacci (Tab)	$O(n)$	$O(n)$	$O(1)$
0/1 Knapsack	$O(n \times W)$	$O(n \times W)$	$O(W)$
LCS	$O(n \times m)$	$O(n \times m)$	$O(\min(n, m))$
LIS (n^2)	$O(n^2)$	$O(n)$	—
LIS ($n \log n$)	$O(n \log n)$	$O(n)$	—
House Robber	$O(n)$	$O(n)$	$O(1)$
Coin Change	$O(n \times \text{amount})$	$O(\text{amount})$	—
Edit Distance	$O(n \times m)$	$O(n \times m)$	$O(\min(n, m))$
Grid DP (min path sum)	$O(n \times m)$	$O(n \times m)$	$O(m)$

Detailed Breakdown

Algorithm	Preprocessing	Query	Update	Best Case	Worst Case
Fibonacci (Memo)	$O(n)$ cache init	$O(1)$ after precompute	$O(1)$ per state	$O(n)$	$O(n)$
Fibonacci (Tab)	$O(1)$	$O(n)$	$O(1)$ per state	$O(n)$	$O(n)$
0/1 Knapsack	$O(n \times W)$ init	$O(1)$	$O(1)$ per cell	$O(n \times W)$	$O(n \times W)$
LCS	$O(n \times m)$ init	$O(1)$	$O(1)$ per cell	$O(n \times m)$	$O(n \times m)$
LIS (n^2)	$O(n)$ init	$O(1)$	$O(n)$ per state	$O(n^2)$	$O(n^2)$

Algorithm	Preprocessing	Query	Update	Best Case	Worst Case
LIS (n log n)	O(1)	O(1)	O(log n) per element	O(n log n)	O(n log n)
Coin Change	O(amount) init	O(1)	O(n) per state	O(n × amount)	O(n × amount)

Key Observations

- **Memoization** adds O(n) recursion stack space in the worst case.
- **Tabulation** can be optimized to O(1) or O(n) space when the recurrence only depends on a few previous states.
- **2D DP** space can often be reduced from O(n²) to O(n) by keeping only the previous row.
- **LIS** can be solved in O(n log n) using binary search (patience sorting), which is not pure DP but is the standard optimal approach.

12. Common Patterns

Pattern 1: Linear DP (1D)

How to identify: The problem involves a sequence where $dp[i]$ depends on a few previous $dp[j]$ values ($j < i$).

General approach: Define $dp[i]$ = best answer for prefix of length i. Transition uses $dp[i-1]$, $dp[i-2]$, etc.

Examples:

- Fibonacci numbers
- House Robber / House Robber II
- Climbing Stairs
- Min Cost Climbing Stairs
- Decode Ways

Pattern 2: Knapsack DP

How to identify: You have items with weights and values, and a capacity constraint. Each item can be used 0/1 time (0/1 Knapsack) or unlimited times (Unbounded Knapsack).

General approach:

- 0/1 Knapsack: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])$
- Unbounded: $dp[i][w] = \max(dp[i-1][w], dp[i][w-wt[i]] + val[i])$ (forward traversal)
- 1D optimization: traverse backwards for 0/1, forwards for unbounded.

Examples:

- Subset Sum (0/1 Knapsack variant)
- Partition Equal Subset Sum
- Coin Change (unbounded)

- Target Sum
- Ones and Zeroes

Pattern 3: LIS (Longest Increasing Subsequence)

How to identify: Find longest subsequence (not necessarily contiguous) that is increasing/decreasing/non-decreasing.

General approach:

- $O(n^2)$: $dp[i] = 1 + \max(dp[j])$ for all $j < i$ with $arr[j] < arr[i]$
- $O(n \log n)$: Patience sorting with binary search on tails array.

Variations:

- Longest Decreasing Subsequence (reverse the array)
- Longest Bitonic Subsequence (LIS from left + LIS from right)
- Number of Longest Increasing Subsequences
- Russian Doll Envelopes (2D LIS)

Pattern 4: LCS (Longest Common Subsequence)

How to identify: Compare two strings/arrays to find the longest common subsequence.

General approach: $dp[i][j]$ = LCS of first i chars of A and first j of B.

Variations:

- Shortest Common Supersequence ($n + m - \text{LCS}$)
- Longest Palindromic Subsequence (LCS of string with its reverse)
- Edit Distance (LCS with costs)
- Minimum Insertions to Make Palindrome ($n - \text{LPS}$)

Pattern 5: Grid DP

How to identify: Robot moving in a grid, minimum path sum, unique paths, etc.

General approach: $dp[i][j]$ = best value to reach cell (i, j) . Transition from top and left.

Examples:

- Unique Paths
- Minimum Path Sum
- Dungeon Game
- Triangle (minimum path sum in triangle)
- Maximal Square

Pattern 6: Interval DP

How to identify: The problem involves a sequence where the answer for a range $[i, j]$ depends on answers for subranges.

General approach: $dp[i][j]$ = best value for substring/range from i to j. Iterate by increasing length.

Examples:

- Matrix Chain Multiplication
- Palindrome Partitioning
- Burst Balloons
- Stone Game
- Longest Palindromic Substring

Pattern 7: State Machine DP

How to identify: The problem has multiple states (buy/sell, hold/cooldown, etc.) and transitions between them.

General approach: Define $dp[i][state]$ = best value at position i when in a given state.

Examples:

- Best Time to Buy and Sell Stock with Cooldown
- Best Time to Buy and Sell Stock with Transaction Fee
- House Robber III (tree DP with two states)
- Student Attendance Record

Pattern 8: DP on Trees

How to identify: Tree structure where each node's answer depends on its children.

General approach: DFS from leaves to root. For each node, compute dp based on children's dp values.

Examples:

- Diameter of Binary Tree
- House Robber III
- Binary Tree Maximum Path Sum
- Tree DP with two states (include/exclude node)

13. Common Mistakes

● Mistake 1: Wrong Initialization

```
// WRONG: Using INT_MAX for min problems
vector<int> dp(n+1, INT_MAX); // dp[0] = 0 works, but overflow risk

// RIGHT: Use a large but safe value
vector<int> dp(n+1, 1e9);
dp[0] = 0;
```

● Mistake 2: Off-by-One in Indexing

```
// WRONG: dp[n] instead of dp[n-1]
for (int i = 1; i <= n; i++) {
    dp[i] = max(dp[i-1], dp[i-2] + arr[i]); // arr[i] out of bounds!
}

// RIGHT: Use 0-indexed or 1-indexed consistently
for (int i = 1; i < n; i++) {
    dp[i] = max(dp[i-1], (i>=2 ? dp[i-2] : 0) + arr[i]);
}
```

● Mistake 3: Wrong Traversal Order (Space Optimized Knapsack)

```
// WRONG: Forward traversal allows reusing the same item multiple times
for (int w = 0; w <= W; w++) {
    dp[w] = max(dp[w], dp[w - wt[i]] + val[i]); // Unbounded knapsack behavior!
}

// RIGHT: Backward traversal for 0/1 Knapsack
for (int w = W; w >= wt[i]; w--) {
    dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
}
```

● Mistake 4: Forgetting Base Cases

```
// WRONG: No base case for n=0 or n=1
vector<int> dp(n+1);
dp[2] = dp[1] + dp[0]; // Undefined values!

// RIGHT: Always set base cases first
dp[0] = 0;
dp[1] = 1;
```

● Mistake 5: Assuming DP Always Works for Large Constraints

```
// n = 10^5, but O(n^2) DP
// This will TLE. Need a better approach (greedy, binary search, etc.)
```

● Mistake 6: Not Handling the "No Solution" Case

```
// WRONG: Returns dp[amount] = INF (or INT_MAX)
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount+1, INT_MAX);
    dp[0] = 0;
```

```
// ...
return dp[amount]; // Returns INT_MAX if no solution!
}

// RIGHT: Check for sentinel value
return dp[amount] == INT_MAX ? -1 : dp[amount];
```

● Mistake 7: Wrong State Definition

```
// WRONG: State missing information
// Problem: max profit with unlimited transactions, but with cooldown
dp[i] = max(dp[i-1], dp[i-1] + profit[i]); // Missing cooldown state!

// RIGHT: Add state dimension
// dp[i][0] = max profit with no stock on day i
// dp[i][1] = max profit with stock on day i
```

● Mistake 8: Integer Overflow

```
// WRONG: dp[i] may overflow int
int dp[1000];
dp[i] = dp[i-1] + dp[i-2]; // Fibonacci at n=50 overflows int

// RIGHT: Use long long
long long dp[1000];
// Or use modulo if required
dp[i] = (dp[i-1] + dp[i-2]) % MOD;
```

● Mistake 9: Memoization with Global/Static Arrays Not Being Reset

```
// WRONG: memo is initialized once for the first call only
int solve(int n) {
    static vector<int> memo(1000, -1); // Fine for single test case
    // ...
}

// RIGHT: Reinitialize for each test case
vector<int> memo(n+1, -1);
int solve(int n, vector<int>& memo) {
    // ...
}
```

● Mistake 10: Forgetting to Handle Edge Cases

```
// WRONG: No check for empty input
int rob(vector<int>& nums) {
    int n = nums.size();
    dp[0] = nums[0]; // Out of bounds if n == 0!
    // ...
}

// RIGHT: Guard against empty input
if (n == 0) return 0;
if (n == 1) return nums[0];
```

14. Edge Cases

General Edge Cases for All DP Problems

Edge Case	What to Check	Example
Empty input	$n = 0$	<code>rob({})</code> should return 0
Single element	$n = 1$	<code>fib(0)</code> or <code>fib(1)</code> should return correctly
Two elements	$n = 2$	Transition should work with only 2 base elements
All equal	No increasing/decreasing possible	LIS of <code>[5,5,5]</code> = 1
Already sorted	LIS should be n	<code>[1,2,3]</code> → LIS = 3
Reverse sorted	LIS should be 1	<code>[3,2,1]</code> → LIS = 1
Large values	Need <code>long long</code>	Knapsack with values up to 10^9
Negative values	Max/min problems	Max subarray sum with negative numbers
Zero values	Division by zero?	Coin change with coin 0 (invalid)
Integer overflow	Use <code>long long</code> or modulo	Fibonacci(50) exceeds int range

Problem-Specific Edge Cases

Problem	Edge Case	Expected Behavior
0/1 Knapsack	$W = 0$	Return 0 (no capacity)
0/1 Knapsack	All items weigh more than W	Return 0 (nothing fits)
LCS	One string empty	Return 0
LCS	Both strings identical	Return n (length of string)

Problem	Edge Case	Expected Behavior
LCS	No common characters	Return 0
Coin Change	<code>amount = 0</code>	Return 0 (no coins needed)
Coin Change	No combination possible	Return -1
House Robber	All houses have 0 money	Return 0
Edit Distance	One string empty	Return length of the other string
Edit Distance	Both strings identical	Return 0
Grid DP	Single cell grid	Return that cell's value
Grid DP	Single row or column	Only one path, straightforward DP

15. Variations

Variation 1: 0/1 Knapsack → Unbounded Knapsack

What changes: Each item can be used unlimited times.

Transition difference:

- 0/1 Knapsack: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])$
- Unbounded: $dp[i][w] = \max(dp[i-1][w], dp[i][w-wt[i]] + val[i])$

Space optimized direction:

- 0/1: traverse W backwards
- Unbounded: traverse W forwards

Importance: High — Coin Change is unbounded knapsack.

Variation 2: LIS → Number of LIS / Longest Bitonic Subsequence

What changes: Track count of LIS ending at each index, or compute LIS from both directions.

When used: Problems asking "how many LIS exist?" or "longest mountain subsequence".

Importance: Medium — appears in placements.

Variation 3: LCS → Shortest Common Supersequence / Longest Palindromic Subsequence

What changes: $SCS = n + m - LCS$. $LPS = LCS$ of string and its reverse.

When used: String manipulation problems.

Importance: Medium — good to know the connections.

Variation 4: 1D DP → State Machine DP

What changes: Add a dimension for state (holding stock vs not, cooldown vs not).

Example:

```
dp[i][0] = max profit, no stock on day i
dp[i][1] = max profit, holding stock on day i
```

Importance: High — stock problems are extremely common in placements.

Variation 5: 2D DP → Interval DP

What changes: `dp[i][j]` represents a range from `i` to `j`. Iterate by increasing length.

Example: Matrix Chain Multiplication: `dp[i][j] = min(dp[i][k] + dp[k+1][j] + cost)`

Importance: High — burst balloons, palindrome partitioning are classic.

Variation 6: DP on Grid → DP with Obstacles / Varying Costs

What changes: Add obstacle checks or cost conditions.

Example: Unique Paths II (with obstacles) — skip cells with obstacles.

Importance: Medium — common in interviews.

Variation 7: DP with Bitmask

What changes: State is a bitmask representing which items are used. Used for TSP, assignment problems.

Example: `dp[mask][i]` = min cost to visit cities in `mask`, ending at city `i`.

Importance: Medium — needed for CP and some hard interview problems.

Variation 8: Digit DP

What changes: DP on digits of a number. Count numbers with certain properties in a range.

Example: Count numbers in `[L, R]` with sum of digits = `S`.

Importance: Medium-High — common in CP, appears in some placements.

16. Related Algorithms/Data Structures

Memoization vs Tabulation

Aspect	Memoization	Tabulation
Direction	Top-down ($n \rightarrow 0$)	Bottom-up ($0 \rightarrow n$)
Implementation	Recursive + cache	Iterative + table
Space	Recursion stack + cache	Table only

Aspect	Memoization	Tabulation
Performance	Slower (recursion overhead)	Faster (no recursion)
Ease of writing	More natural	Requires ordering
Stack overflow	Possible for deep recursion	Not possible
Space optimization	Harder	Easier

DP vs Greedy

Aspect	DP	Greedy
Decision	Explores all options	Makes one locally optimal choice
Correctness	Always correct if state is right	Requires proof of optimal substructure
Complexity	Usually $O(n^2)$ or more	Usually $O(n \log n)$ or less
When to use	When greedy fails	When greedy is proven correct

DP vs Divide & Conquer

Aspect	DP	Divide & Conquer
Subproblems	Overlapping	Independent
Storage	Caches results	No caching
Example	Fibonacci	Merge Sort
Complexity reduction	Exponential $\rightarrow$ Polynomial	$O(n \log n)$ typical

DP vs BFS/DFS (for shortest path)

Problem	Best Approach
Shortest path in unweighted graph	BFS
Shortest path in weighted graph (no negative)	Dijkstra (greedy + DP)
Shortest path with negative weights	Bellman-Ford (DP)
All-pairs shortest path	Floyd-Warshall (DP)

DP vs Binary Search

- **LIS:** $O(n^2)$ DP or $O(n \log n)$ with binary search (patience sorting).
- **DP with binary search optimization:** When transition involves finding a max/min in a range, use binary search + segment tree/BIT.

DP vs Matrix Exponentiation

- **Fibonacci:** $O(n)$ DP or $O(\log n)$ with matrix exponentiation.

- **Linear recurrences:** Use matrix exponentiation when n is very large (10^{18}).

17. Practice Problems

Easy

#	Problem	Platform	Main Idea	Difficulty
1	Climbing Stairs	LeetCode	1D DP, Fibonacci-like	Easy
2	House Robber	LeetCode	1D DP with adjacent constraint	Easy

Medium

#	Problem	Platform	Main Idea	Difficulty
1	Coin Change	LeetCode	Unbounded knapsack, min coins	Medium
2	Longest Increasing Subsequence	LeetCode	LIS, $O(n^2)$ and $O(n \log n)$	Medium
3	0/1 Knapsack	GFG	Classic 2D DP	Medium
4	Edit Distance	LeetCode	2D DP, string comparison	Medium
5	Longest Common Subsequence	LeetCode	2D DP, string comparison	Medium

Hard

#	Problem	Platform	Main Idea	Difficulty
1	Burst Balloons	LeetCode	Interval DP, tricky transition	Hard
2	Best Time to Buy and Sell Stock IV	LeetCode	State machine DP, k transactions	Hard
3	Distinct Subsequences	LeetCode	2D DP, counting	Hard
4	Minimum Cost to Cut a Stick	LeetCode	Interval DP	Hard
5	Shortest Path Visiting All Nodes	LeetCode	DP with bitmask (TSP variant)	Hard

Additional CP Problems

#	Problem	Platform	Main Idea
1	Dice Combinations	CSES	1D DP, counting ways
2	Minimizing Coins	CSES	Coin change, classic
3	Removing Digits	CSES	1D DP, digit manipulation
4	Grid Paths	CSES	2D DP on grid
5	Edit Distance	CSES	Classic 2D DP
6	Longest Increasing Subsequence	CSES	LIS, $O(n \log n)$

#	Problem	Platform	Main Idea
7	Array Description	CSES	2D DP, counting with constraints

18. Interview Explanation

What to Say When Asked "What is Dynamic Programming?"

"Dynamic Programming is a technique for solving optimization and counting problems by breaking them into overlapping subproblems, solving each subproblem once, and storing the result for reuse.

The key prerequisites are two properties:

1. **Optimal Substructure** — The optimal solution of the problem can be built from optimal solutions of its subproblems.
2. **Overlapping Subproblems** — The same subproblems appear multiple times, so caching saves work.

There are two main approaches:

- **Top-Down (Memoization):** Start with a recursive function and add a cache. Before computing, check if the result is already cached. If yes, return it; if no, compute recursively and cache.
- **Bottom-Up (Tabulation):** Start with base cases and iteratively fill a table from smallest to largest subproblems.

The most important part of any DP solution is defining the **state** — what does `dp[i]` or `dp[i][j]` represent? Once you have the state, the **transition** (recurrence relation) usually follows naturally.

For example, in the House Robber problem, the state is `dp[i]` = maximum money we can rob from the first *i* houses. The transition is `dp[i] = max(dp[i-1], dp[i-2] + nums[i])` — either skip house *i* or rob it and add to the best from two houses ago.

Common DP patterns include 1D DP (Fibonacci, House Robber), 2D DP (LCS, Edit Distance, Knapsack), interval DP (Burst Balloons), and state machine DP (stock problems)."

What to Say When Asked "How Do You Decide Between Memoization and Tabulation?"

"I prefer **tabulation** for most interview problems because it avoids recursion overhead and stack overflow, and it's easier to optimize for space. But I use **memoization** when the state space is sparse — for example, when not all states are reachable — or when the recursion is more natural to write and I'm confident the depth won't overflow the stack.

In practice, I often start with memoization to think through the problem, then convert to tabulation for the final solution if needed."

19. Revision Notes

Quick Reference

Concept	Key Point
DP	Solve subproblems once, store, reuse
Memoization	Top-down + cache. Check cache before computing.
Tabulation	Bottom-up + table. Fill from smallest to largest.
State	Parameters that uniquely identify a subproblem
Transition	Formula relating <code>dp[state]</code> to smaller states
Base Case	Smallest subproblem, solved directly
Space Optimization	Keep only last few rows/values

Formula Reminders

Problem	Transition
Fibonacci	<code>dp[i] = dp[i-1] + dp[i-2]</code>
0/1 Knapsack	<code>dp[i][w] = max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])</code>
Unbounded Knapsack	<code>dp[w] = max(dp[w], dp[w-wt[i]] + val[i])</code> (forward)
LCS	if match: <code>1 + dp[i-1][j-1]</code> , else: <code>max(dp[i-1][j], dp[i][j-1])</code>
LIS (n^2)	<code>dp[i] = 1 + max(dp[j])</code> for <code>j < i</code> and <code>arr[j] < arr[i]</code>
LIS ($n \log n$)	Binary search on <code>tails</code> array
House Robber	<code>dp[i] = max(dp[i-1], dp[i-2] + nums[i])</code>
Coin Change	<code>dp[i] = min(dp[i], 1 + dp[i-coin])</code> for each coin
Edit Distance	if match: <code>dp[i-1][j-1]</code> , else: <code>1 + min(delete, insert, replace)</code>

Template Reminder

```
// 1D DP template
vector<int> dp(n+1, init_val);
dp[0] = base0; // or dp[0..k] = base
for (int i = 1; i <= n; i++) {
    dp[i] = f(dp[i-1], dp[i-2], ...);
}

// 2D DP template
vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
// Base cases
for (int i = 0; i <= n; i++) dp[i][0] = ...;
for (int j = 0; j <= m; j++) dp[0][j] = ...;
// Fill
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        dp[i][j] = f(dp[i-1][j], dp[i][j-1], dp[i-1][j-1], ...);
    }
}
```

```

    }
}

// Space optimized 2D DP
vector<int> prev(m+1, base), curr(m+1, 0);
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        curr[j] = f(prev[j], curr[j-1], prev[j-1], ...);
    }
    swap(prev, curr);
}

```

Common Traps

Trap	Fix
Off-by-one in indexing	Use 1-indexed DP with 0-indexed arrays carefully
Integer overflow	Use <code>long long</code>
Missing base cases	Always handle $n=0$, $n=1$
Wrong traversal order for 0/1 Knapsack	Backwards for 0/1, forwards for unbounded
INF value too large causing overflow	Use <code>1e9</code> instead of <code>INT_MAX</code>
Forgetting to return -1 for no solution	Check sentinel value before returning
Memoization not reset for multiple test cases	Reinitialize per test case

20. Final Cheat Sheet

When to Use DP

Does the problem ask for:

- Maximum/minimum something?
- Number of ways to do something?
- Is it possible to achieve something?
- Longest/shortest subsequence?

Can you write a brute-force recursion?

Does the recursion revisit the same states?

If YES to all → DP is your answer.

Main Operations

Operation	Code
Memoization init	<code>vector<int> memo(n+1, -1);</code>
Memoization check	<code>if (memo[i] != -1) return memo[i];</code>
Memoization store	<code>return memo[i] = f(...);</code>
Tabulation init	<code>vector<int> dp(n+1, 0);</code>
Tabulation base	<code>dp[0] = 0; dp[1] = 1;</code>
Tabulation fill	<code>for (int i = 2; i <= n; i++) dp[i] = ...</code>
Space opt (1D → O(1))	Keep 2-3 variables, rotate
Space opt (2D → 1D)	<code>prev</code> and <code>curr</code> arrays

Complexity Quick Reference

Pattern	Time	Space (Full)	Space (Opt)
1D Linear DP	$O(n)$	$O(n)$	$O(1)$
1D DP with inner loop	$O(n^2)$	$O(n)$	$O(n)$
2D DP (strings)	$O(n \times m)$	$O(n \times m)$	$O(\min(n, m))$
2D DP (knapsack)	$O(n \times W)$	$O(n \times W)$	$O(W)$
Interval DP	$O(n^3)$	$O(n^2)$	$O(n^2)$
DP with bitmask	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$	$O(n \cdot 2^n)$

Key Code Idea

```
// The DP trinity: State, Transition, Base Case
int solve() {
    // 1. Define state
    vector<int> dp(n + 1, 0);

    // 2. Base cases
    dp[0] = 0;
    dp[1] = 1;

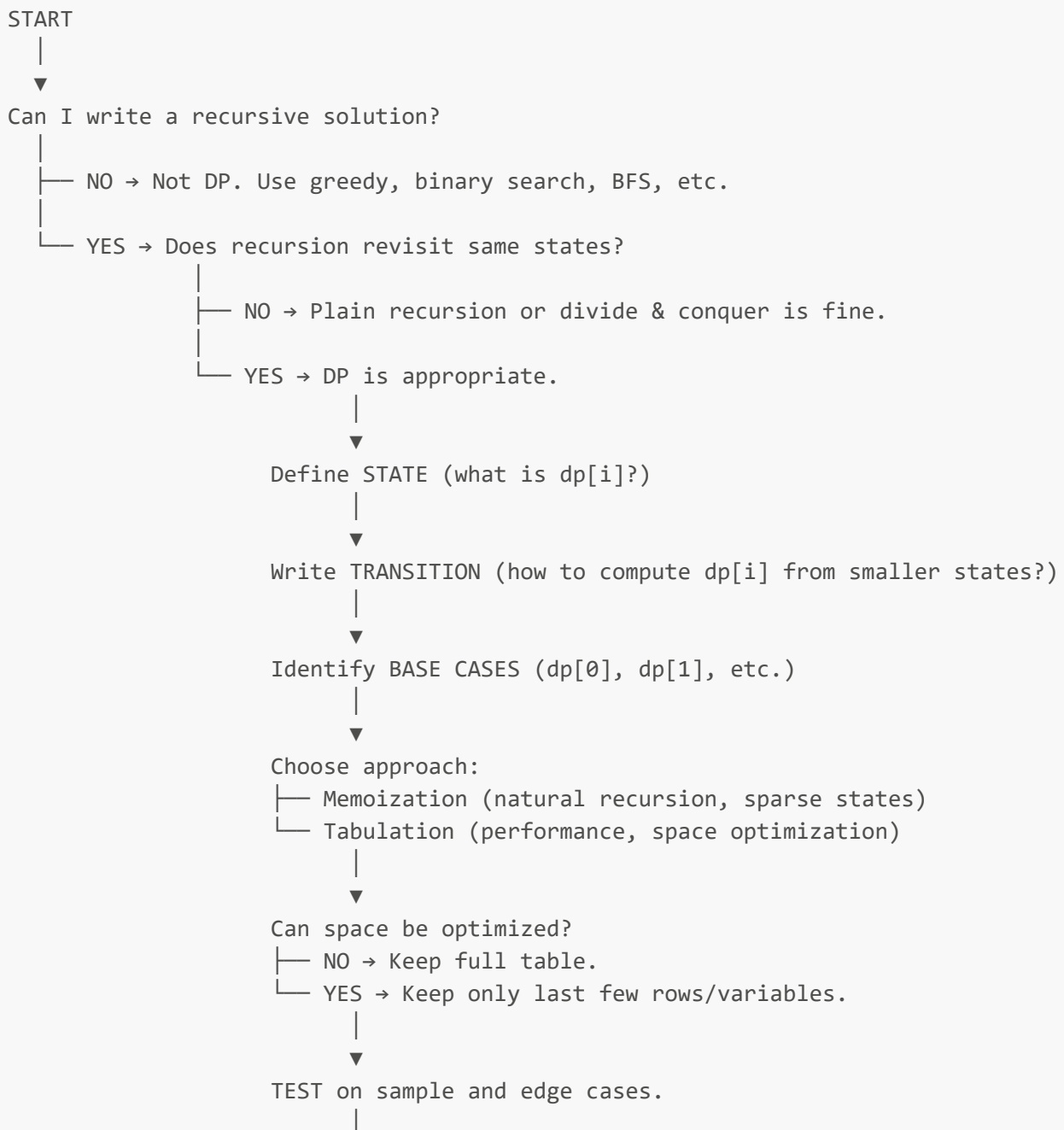
    // 3. Transition (fill order matters)
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2]; // Example: Fibonacci
    }

    // 4. Answer
    return dp[n];
}
```

⚠ Important Edge Cases

- Empty input ($n = 0$)
- Single element ($n = 1$)
- All elements equal / no variation
- Already sorted / reverse sorted
- No solution exists (return -1)
- Integer overflow (use long long)
- Capacity less than all items (knapsack = 0)
- Amount = 0 (coin change = 0)
- Strings of different lengths (LCS, edit distance)
- Large constraints ($n = 10^5$, need $O(n \log n)$ or better)

🏆 Problem-Solving Flowchart



▼
DONE ✓

Pro Tip: The single most important skill for DP is **state definition**. If you can define $dp[i]$ or $dp[i][j]$ correctly, the transition usually writes itself. Practice on 20-30 problems and the patterns will become automatic.